

Entropy-Based Two-Phase Optimization Algorithm for Solving Wordle-like Games

Yen-Chi Chen
Academia Sinica, Taiwan
Email: zxkyjimmy@gmail.com

Hao-En Kuan
NYCU, Taiwan
Email: kuanhaoan@gmail.com

Yen-Shun Lu
NYCU, Taiwan
Email: eric890923@gmail.com

Tzu-Chun Chen
NYCU, Taiwan
Email: tzuchunchen1015@gmail.com

I-Chen Wu
NYCU, Taiwan
Email: icwu@cs.nycu.edu.tw

Abstract—In the past, a method called Two-Phase Optimization Algorithm (TPOA) was designed by Shan-Tai Chen to solve the game of Mastermind and the AB game efficiently. In this paper, we proposed a modified version called Entropy-Based TPOA (EBTPOA) for Wordle-like games. It is a combination of his algorithm and our previous work on Nerdle with greedy method. It focuses on not only effectiveness but also efficiency while finding optimal results. In Wordle-like games, EBTPOA performs better with fewer guess times on average than TPOA.

In Wordle, EBTPOA hits the answer optimally within 3.42117 times on average, and 5 times in the worst case, and the best opening word is "SALET". In Nerdle, EBTPOA hits the answer within 3.01947 times on average, and 4 times in the worst case, and the best opening equation is "52-34=18". These are the best results up to date, and particularly none has achieved the result for Nerdle so far.

As for efficiency, by expanding with fewer branches and limiting the depth of exploration, EBTPOA can obtain the optimal result with a lower time complexity compared to related works.

Keywords—Wordle, Nerdle, Deductive Game, Entropy, Search Strategy, Two-Phase Optimization Algorithm

I. INTRODUCTION

As of January 2022, a puzzle game, Wordle [1], quickly caught attention worldwide. Wordle is a word-guessing game. Players can type in a valid word with five letters in each round, and then the program will respond with the original input colored. A gray cell indicates the alphabet does not occur in the answer word; a yellow cell indicates the alphabet exists in the answer but is misplaced; a green cell means the alphabet appears at the exact position. Furthermore, there are only six chances to guess the word.

In Figure 1, there is an example of a game in Wordle. Suppose the correct answer is "WHOO", and we guess "BOOKY". The second "O" in "BOOKY" will turn green, since this character appears at the exact same place as the first "O" in "WHOO". Besides, we will get a yellow cell at the position of the first "O" of "BOOKY" because this character appears in "WHOO" but has incorrect position. Finally, all

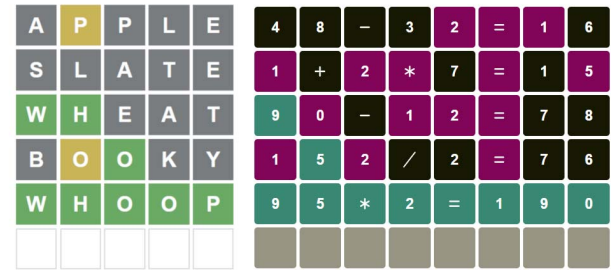


Fig. 1. Examples of a game in Wordle and Nerdle. Purple and black cells in Nerdle are similar to yellow and gray cells in Wordle, respectively.

TABLE I
SIZES OF THE GAME LIST AND THE LEGAL LIST IN DIFFERENT GAMES

List	Game	Size
Game List	Wordle	2315
	Nerdle	17723
Legal List	Wordle	12972
	Nerdle	40346

the other cells of "BOOKY" will be colored gray, since those characters do not appear in "WHOO". Also, each character in the correct answer can only be considered once; that's why the second "P" in "APPLE" is colored gray.

In addition to Wordle, some similar games, such as Nerdle [2], were introduced soon after Wordle was presented. The rules in Nerdle and Wordle are almost identical, except for the inputs being changed from five-letter words to eight-symbol equations as shown in Figure 2.

When we consider each next action, there are 2 lists for us to choose from. One is the game list, which is a list of guesses that are possible to be answers when playing the game; the other is the legal list, and it contains all acceptable inputs. Both were tried in our experiments and the sizes of them are listed in Table I.

The goal of this paper is to find a way to solve these Wordle-like games in a general way. In the past, a method called Two-Phase Optimization Algorithm (TPOA) [3] was designed to solve the game of Mastermind and the AB game efficiently. In this paper, we propose a modified method called Entropy-Based TPOA (EBTPOA) for Wordle-like games. In Wordle, EBTPOA hits the answer optimally within 3.42117 times on average, and 5 times in the worst case, and the best opening word is "SALET". In Nerdle, EBTPOA hits the answer within 3.01947 times on average, and 4 times in the worst case, and the best opening equation is "52-34=18". These are the best results up to date, and particularly none has achieved the result for Nerdle so far.

II. RELATED WORKS

A. 3Blue1Brown's Entropy Method

3Blue1Brown [4], also known as 3B1B, proposed a method utilizing entropy to solve Wordle, which pursues the largest entropy in each round. The idea is to get more expected information and let the remaining uncertainty be as low as possible. Besides, he calculated the expected information two steps further to get the max entropy combining any 2 different words. By this method, he could guess the answer with an average of 3.42117 tries and at most 5 tries in Wordle.

B. Jonathan's Method

Jonathan introduced methods using decision trees and heuristics [5]. His methods can solve 98 percent of words in 4 guesses and about 3.42117 on average. Moreover, his heuristics are based on the size of the partitioned sets. In other words, if words have the same response under the specific answer, he uses a heuristic to classify those words into the same partitioned set. For example, if our guess is "TRACE", we will have one set of words where the first letter is gray, and the rest are green ("BRACE", "GRACE"). Therefore, Jonathan's method will gain more information when most partitioned sets are small.

C. Dimitris' and Alex's Research

On September 2nd, 2022, Doctor Dimitris and his doctoral student Alex published a paper [6] claiming that they had found the optimal result in Wordle. In their research, they proposed a framework based on Exact Dynamic Programming to solve Wordle. They first derived a mathematical model of the game, constructed a Bellman Equation, and then applied some optimizations and parallelization to make the problem tractable. Despite that, it still costs $O(B^D)$ for their algorithm to expand their Bellman equation, where B is all legal words and D is the maximum allowed tries in the game. In the end, the best opening word they found was "SALET" and the lowest average guess times was 3.42117. We will show that our best result was the same as the optimal one they found.

D. Two-Phase Optimization Algorithm (TPOA)

Two-Phase Optimization Algorithm [3] was introduced by Shan-Tai Chen to solve the AB game and Mastermind,

which are similar to Wordle and Nerdle. TPOA tries to reduce the number of expanded nodes during the exploration of the game tree as much as possible with the help of his heuristic evaluation, and the process is split into an exploration phase and an exploitation phase.

Before elaborating on these phases, we have to know that there are 2 versions of TPOA, the first one is $TPOA^+$ and the second one is $TPOA^*$. In $TPOA^+$, we specify a branching factor b and an exploration depth d . Before the searching depth deepens beyond d , each time, it only expands the top b promising nodes in the exploration phase. After the depth goes beyond d , it enters the exploitation phase, i.e. it only expands one node until reaching the answer. In $TPOA^*$, it takes the idea of "annealing", i.e. the number of expanded nodes is reduced as the searching depth increases. This way, it can further reduce the number of expanded nodes without sacrificing the quality of the found strategy.

To find the first b promising branches, he introduced Hash Collision Group (HCG) to group guesses (which are named "codes" in Mastermind in his case) with similar effects by putting guesses with the same hash value together. Then, we can first pick b promising groups and later pick one representative from each picked group.

According to Chen's paper, TPOA is able to find the same results as what a brute-force search found as long as b and d are fine-tuned. Moreover, since Wordle and Nerdle are similar to Mastermind and the AB game, we determined to adopt his algorithm and adjust it to solve Wordle-like games.

III. ALGORITHMS

In this section, we will introduce two types of algorithms, the greedy method and the search method, to solve Wordle-like games. The former is our previous work [7], while the latter is our improvement this time. Before we run into our algorithms, we will first elaborate on the structure of the game tree. As shown in Figure 2, the guessing process can be represented by a game tree composed of state nodes and chance nodes. Each state node stands for a set of remaining possible answers, while a chance node represents a guess and it derives many possible responses where each can further derive a state node that represents a smaller set of remaining possible answers.

When the state node has only one word/equation inside, it must be the answer and we will finish the game in the next guess. This kind of state node is our goal, and its level is the time needed to hit the answer. Therefore, in order to hit the answer with as few guess times as possible in the average case and in the worst case, we tried two types of methods which are the greedy method and the search method.

A. Greedy Method

The idea of the greedy method is to maximize the entropy of the distribution of possible responses in each round. We want to find a guess that leads to the flattest distribution. In

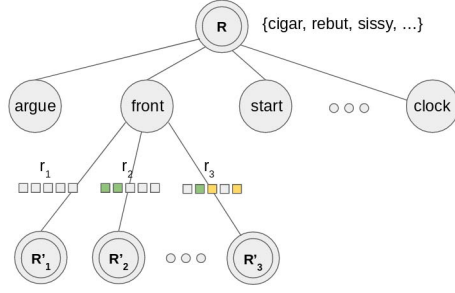


Fig. 2. Example of a game tree

- Double circled nodes are state nodes, representing a set of remaining possible answers.
- Single circled nodes are chance nodes, representing a chosen guess.
- Squares are possible responses after choosing some guess, and each response with a chosen guess can be used to filter the original set.

this way, whatever response we receive after a guess, we will not prune too few possible answers.

In our previous work [7], we utilized the heuristic of entropy [8] to help us decide the next action. We proposed 4 versions, and each version was an improvement from the previous one. Here, we introduce the G-V1 and G-V4 in the paper, since we will combine G-V1 with TPOA and compare it with our last version of greedy method.

1) G-V1

Same as 3B1B's method using entropy, each time, it takes the remaining possible answers into consideration and finds the one contributing the largest entropy as the next action.

2) G-V4

In each round, it takes all guesses into consideration. However, if the remaining possible answers are few enough, it will only consider them. Besides, there's another rule. Once the maximum entropy of a set of remaining possible answers is less than a threshold value, which is the entropy of a distribution where all of the guesses cause the same response except the answer, the solver then takes all guesses into consideration again. By choosing a guess outside of this set, it may be able to eliminate more possibilities from the set.

B. Search Method

In order to improve our previous work on greedy method, we tried a search-based method, TPOA. Aside from using the original version of it, we introduced the heuristic of our greedy methods to TPOA, which became our Entropy-based TPOA (EBTPOA). The differences were that we adjusted the candidate selection and the estimation of left guess times. To elaborate, we prioritized HCGs with their entropy and picked b representatives. Besides, when entering the exploitation phase, instead of assuming it would guess each remaining possible

answer one by one until hitting the real answer, we ran a simulation with G-V1 to estimate more accurate expected left guess times. Both of these 2 modifications will be explained in the following 2 sections.

1) Candidate Selection

In TPOA, there are two hash functions, $Hash_e$ and $Hash_w$, used to classify all words into different groups. When some words have the same hash value (namely, have similar response distributions after guessing), they are put into the same HCG. $Hash_e$ is designed for the expected case, and it will sort the number of times for each response and ignore what the actual response is. $Hash_w$ is designed for the worst case, so it only cares how many different responses there are after a word is guessed. For example, if a player guesses a word and has a chance to get three different responses, each of which appears 1, 3, 2 times, respectively, then $Hash_e = (3, 2, 1)$ and $Hash_w = 3$.

These two hash functions are good for grouping. However, they can neither evaluate which group is good enough to be selected first nor know which word can stand for its group. In order to solve the above problems, we proposed $Hash_E$. In $Hash_E$, we considered the group with the largest entropy first, then the group with the second largest entropy, and so on. In each considered group, we would pick at most 2 representatives, the one that is also a possible answer and the one that is not. It's possible that one of them did not exist. In that case, we simply picked one representative from it. With this in mind, we repeated it until we had enough candidates, i.e. the value of the branching factor b .

It is worth mentioning that by picking 2 types of representatives, it would be good at overcoming the problem that the remaining possible answers are all similar. For example, there are "TRAIN", "BRAIN", "DRAIN", and "GRAIN" left, and the answer is "DRAIN". It will cost us 4 guesses in the worst case if we only consider those words. However, if we bring words outside this set of words as candidates, it will probably help us quickly reduce the number of possible answers.

2) Simulation

As mentioned above, the other part where we introduced entropy to TPOA is the estimation of left guess times when entering the exploitation phase. During the simulation, we applied the strategy of G-V1, i.e. in each round, we would choose a guess contributing the most entropy. The reason to use version 1 is that it's simple and effective enough as a heuristic to find a good guess. In addition, since we tend to choose guesses from groups that contribute lots of entropy in candidate selection, it's reasonable to run a simulation with G-V1.

As for details of all versions of the search methods we tried, they are shown in Table II. We had 2 objectives, the one that pursues the fewest guess times in the expected case

TABLE II
SETUPS OF EACH SEARCH METHOD

Target	Method	Simulation	Hash Function	(b, d)
Expected	TPOA		$Hash_e$	(10, 2)
	EBTPOA [^]	V	$Hash_e$	(10, 2)
	EBTPOA [#]		$Hash_E$	(10, 2)
	EBTPOA	V	$Hash_E$	(10, 2)
Worst	TPOA		$Hash_w$	(10, 2)
	EBTPOA [^]	V	$Hash_w$	(10, 2)
	EBTPOA [#]		$Hash_E$	(10, 2)
	EBTPOA	V	$Hash_E$	(10, 2)

and the one that purses the fewest guess times in the worst case. Besides, we not only tried TPOA but also EBTPOA and 2 slightly different versions of it, which are EBTPOA[^] and EBTPOA[#]. The former adopted simulation only, while the latter adopted $Hash_E$ only. As for the branching factor and exploration depth, we chose $(b, d) = (10, 2)$.

C. Experiment Methods

As Algorithm 1 shows, we did experiments on finding the expected case and the worst case of a strategy. We tried different setups by switching the *choose_action()* in Algorithm 1 with a specified method. Then, we applied it to each word/equation that can be an answer in the game, so we could calculate each word/equation's guess time. In the end, we were able to obtain the average guess times and the maximum guess times.

Algorithm 1 Pseudocode of Experiment Procedure

```

1: procedure EXPERIMENT
2:    $total \leftarrow 0$ 
3:    $worst \leftarrow 0$ 
4:   for each  $answer \in answers$  do
5:      $times \leftarrow 0$ 
6:     repeat
7:        $chosen \leftarrow choose\_action()$ 
8:        $times \leftarrow times + 1$ 
9:     until  $chosen = answer$ 
10:     $total \leftarrow total + times$ 
11:     $worst \leftarrow \max(worst, times)$ 
12:  end for
13:   $expected \leftarrow \frac{total}{|answers|}$ 
14: end procedure

```

As for our search method, we chose TPOA*. We selected TPOA* instead of TPOA⁺ because the number of branches in TPOA* is fewer than that in TPOA⁺. That is, TPOA* can perform almost the same as TPOA⁺ with less searching time. As a result, we implemented our search method based on TPOA*.

IV. RESULTS

After numerous experiments were done, we discussed some topics: (1) the performance in the expected case and

the worst case, (2) the performance with candidates only from the game list versus the performance with those from the legal list, (3) the effects of branching factor and exploration depth, and (4) performance between our methods and others' from the Internet. These are shown in the following sections.

A. Targets

First, our search methods can be roughly classified into the one that pursues the lowest expected guess times, and the one that pursues the lowest maximum guess times. In other words, we are looking for the strategies performing the best in the expected case and the worst case.

1) Expected Case

G-v4 performed the best among our 4 greedy methods. Later, we tried the search method in order to find better results. As Table III to Table VI shows, our search methods targeting optimizing expected case found strategies with even fewer guess times on average.

To verify our EBTPOA performs better than the original TPOA, we ran different combinations of setups (i.e. TPOA, EBTPOA[^], EBTPOA[#], and EBTPOA). In the end, we found that EBTPOA performed the best on average guess times. As we can see in Table IV, we reached the best results with 3.42117 guesses on average which is the same as what 3Blue1Brown and Jonathon found in Wordle. Besides, in this paper [6], it had proved that 3.42117 was the optimal solution for Wordle on average. Moreover, all our opening words were "SALET", which meant we had already got the optimal solution for Wordle. In addition, in Table VI, we also reached 3.01947 on average, which was the lowest expected guess times in Nerdle at present and the opening equation is "52-34=18". The previous best human strategy was an average of 3.449 times and a worst of 5 times.[9] These results showed our last improved search method (EBTPOA) worked better than greedy methods and original search methods.

2) Worst Case

In the worst case, similarly, all versions of the search methods with different setups (i.e. TPOA, EBTPOA[^], EBTPOA[#], and EBTPOA) could even obtain better results than the greedy method.

What is worth noticing is that when we applied the search method on Wordle, it was possible for us to hit the answer in 5 times even in the worst case, which was the same as the best result that Jonathon proposed. As for Nerdle, all the search methods targeting optimizing the worst case could obtain the results of hitting the answer in 4 times even in the worst case. This time, both the original TPOA and EBTPOA could reach the lowest maximum guess times in either game.

3) EBTPOA vs. TPOA

With the advantage of entropy method, we can select candidate more precisely when running the simulation.

TABLE III
SOLVING WORDLE WITH ALL WORDS IN THE GAME LIST AS
CANDIDATES

Target	Method	1	2	3	4	5	6	≥ 7	Avg	Max
N/A	G-v4	1	83	1135	926	159	8	3	3.51620	7
Expected	TPOA	1	73	1053	1135	52	1	0	3.50410	6
	EBTPOA [^]	1	72	1077	1100	64	1	0	3.49978	6
	EBTPOA [#]	1	76	1162	1026	50	0	0	3.45270	5
	EBTPOA	1	85	1182	978	66	3	0	3.44579	6
Worst	TPOA	1	33	901	1262	118	0	0	3.63197	5
	EBTPOA [^]	1	39	936	1221	114	4	0	3.61339	6
	EBTPOA [#]	1	37	866	1305	106	0	0	3.63844	5
	EBTPOA	1	31	821	1333	129	0	0	3.67300	5

TABLE IV
SOLVING WORDLE WITH ALL WORDS IN THE LEGAL LIST AS
CANDIDATES

Target	Method	1	2	3	4	5	6	≥ 7	Avg	Max
N/A	G-v4	0	22	1117	999	169	8	0	3.57840	6
Expected	TPOA	0	53	1134	1089	39	0	0	3.48121	5
	EBTPOA [^]	1	56	1146	1057	54	1	0	3.47948	6
	EBTPOA [#]	1	68	1214	998	34	0	0	3.43024	5
	EBTPOA	0	78	1225	971	41	0	0	3.42117	5
Worst	TPOA	0	40	948	1244	83	0	0	3.59179	5
	EBTPOA [^]	0	29	941	1263	80	2	0	3.60475	6
	EBTPOA [#]	0	40	911	1302	62	0	0	3.59870	5
	EBTPOA	1	35	903	1300	76	0	0	3.61123	5

For instance, assume we get 2 kinds of distribution, (2, 2, 2) and (3, 1, 1, 1, 1, 1), from 2 candidates. For the first distribution, we have 2 candidates in each set. It's apparently to know that one of them would only need 1 guess to hit the answer, but the other would need 2 guesses. This told us we totally need 3 guesses for each set. Therefore, the average of these 8 candidates would be $(3 \times 4) \div 8 = 1.5$. As for second distribution, those sets with 1 candidate would only need 1 guess for themselves. But for the set with 3 candidates, we need $(1 + 2 + 3) = 6$ guesses to hit all of them in worst case. However, total time of guesses is equal to $6 + (1 \times 5) = 11$, and the average would be $11 \div 8 = 1.375$.

For TPOA, it would select the first one if $Hash_e$ was applied. However, our EBTPOA would select the latter because the entropy of the latter distribution is 1.667 which is larger than the former one whose entropy is only 1.386. According to the above calculation, when TPOA prefers the first distribution which requires 1.5 tries to hit the answer on average, our selection method would choose the second one which only requires 1.375 on average. This result obviously showed the improvement of our EBTPOA. Therefore, in Table III, IV, V, and Table VI we are able to observe that in expected case, results of all varieties of our EBTPOA are better than those of original TPOA. Moreover, our results for worst case could still reach the best result theoretically. This outcome demonstrated that our EBTPOA truly improved the result from original TPOA.

TABLE V
SOLVING NERDLE WITH ALL EQUATIONS IN THE GAME LIST AS
CANDIDATES

Target	Method	1	2	3	4	5	6	≥ 7	Avg	Max
N/A	G-v4	1	1248	14624	1838	12	0	0	3.03453	5
Expected	TPOA	1	1228	14758	1736	0	0	0	3.02855	4
	EBTPOA [^]	1	1238	14751	1733	0	0	0	3.02782	4
	EBTPOA [#]	1	1189	14919	1614	0	0	0	3.02387	4
	EBTPOA	1	1280	14791	1649	2	0	0	3.02093	5
Worst	TPOA	1	386	14473	2863	0	0	0	3.13965	4
	EBTPOA [^]	1	386	14473	2863	0	0	0	3.13965	4
	EBTPOA [#]	1	376	14291	3055	0	0	0	3.15105	4
	EBTPOA	1	376	14291	3055	0	0	0	3.15105	4

TABLE VI
SOLVING NERDLE WITH ALL EQUATIONS IN THE LEGAL LIST AS
CANDIDATES

Target	Method	1	2	3	4	5	6	≥ 7	Avg	Max
N/A	G-v4	1	744	13563	3348	67	0	0	3.15438	5
Expected	TPOA	1	1217	14793	1712	0	0	0	3.02782	4
	EBTPOA [^]	1	1244	14774	1704	0	0	0	3.02584	4
	EBTPOA [#]	1	1173	14980	1569	0	0	0	3.02223	4
	EBTPOA	1	1271	14835	1614	2	0	0	3.01947	5
Worst	TPOA	1	375	14549	2798	0	0	0	3.13660	4
	EBTPOA [^]	1	346	14431	2945	0	0	0	3.14653	4
	EBTPOA [#]	1	366	14359	2997	0	0	0	3.14834	4
	EBTPOA	1	307	14542	2873	0	0	0	3.14467	4

B. Candidate Range

As described before, we tried 2 lists of words as our candidates, which are the game list and the legal list. By comparing Table III with Table IV and Table V with Table VI, we can see that search methods performed even better while greedy methods became worse. It is because the guess chosen by greedy methods was mostly a "sacrificed" one, which means it was definitely not the answer although its entropy was the largest. On the other hand, during candidate selection, search methods always chose a guess that was possible to be an answer and one that was not representative of an HCG. Therefore, it was capable of determining when was the best situation to take a sacrifice move.

C. Branch & Depth

We took the setup of EBTPOA with $(b, d) = (10, 2)$ as our baseline to find out the relationship between one of the factors and the performance. From the experiments, as the branching factor or exploration depth increases, the expected guess times and maximum guess times could be improved. Still, it is not unlimited. For example, in Table VII, we can clearly observe that when the branching factor gradually increased to 10, the performance improved. But after it was beyond 10, we found it converged to a specific value and was unable to get improved from this factor. The same thing happened to the other factor, exploration depth. In Table VII, when the depth was beyond 2, it no longer improved any more.

We speculated that it was because the heuristic of entropy is strong enough and the simulation when entering exploitation phase is accurate enough. Thus, when the exploration depth is

TABLE VII
BRANCHING FACTOR/EXPLORATION DEPTH VS. PERFORMANCE

Game	Candidate	(b, d)	Avg	Max	(b, d)	Avg	Max
Wordle	game	(5, 2)	3.44881	6	(10, 1)	3.44881	6
		(10, 2)	3.44579	6	(10, 2)	3.44579	6
		(15, 2)	3.44579	6	(10, 3)	3.42117	5
	legal	(5, 2)	3.42419	5	(10, 1)	3.42246	6
		(10, 2)	3.42117	5	(10, 2)	3.42117	5
		(15, 2)	3.42117	5	(10, 3)	3.42117	5
Nerdle	game	(5, 2)	3.02251	4	(10, 1)	3.02656	5
		(10, 2)	3.02093	5	(10, 2)	3.02093	5
		(15, 2)	3.02099	5	(10, 3)	3.02099	5
	legal	(5, 2)	3.02071	4	(10, 1)	3.01947	5
		(10, 2)	3.01947	5	(10, 2)	3.01947	5
		(15, 2)	3.01947	5	(10, 3)	3.01947	5

TABLE VIII
COMPARISON OF PERFORMANCE IN WORDLE

Method	1	2	3	4	5	Avg	Max
EBTPOA	0	78	1225	971	41	3.42117	5
3B1B's	0	80	1225	965	45	3.42117	5
Jonathon's	0	78	1223	975	39	3.42117	5
Dimitris' and Alex's	0	93	1230	916	76	3.42117	5

E.g., EBTPOA: $(1 \times 0 + 2 \times 78 + 3 \times 1225 + 4 \times 971 + 5 \times 41) \div 2315 \approx 3.42117$

sufficiently large, it would not improve if the depth was further increased. As for the branching factor, because the best choice was already included in the candidates when the branching factor was sufficiently large, it would not be further improved if other conditions were controlled.

From these results, we knew that EBTPOA with $(b, d) = (10, 2)$ was good enough to meet our expectations, and we realized that neither increasing the branching factor nor enhancing exploration depth was definite to improve the performance. The results would converge eventually.

D. Comparisons

We compared our EBTPOA with 3B1B's best method, Jonathon's method, and Dimitris' and Alex's method. Our methods all reached about 3.42117 guesses on average, and the details are in Table VIII. Although we all found the same result, our EBTPOA is more efficient. Since EBTPOA limits both the branching factor and exploration depth, it is more efficient than Jonathon's method, which only limits the depth of the game tree during the search. Furthermore, both 3B1B and we use entropy to solve Wordle, but we use it to solve Wordle in a different way. What 3B1B did was to expand the top 250 candidate words from the root node and for each candidate, find the one that could lead to the fewest average guess times. However, 3B1B did not use any method similar to HCG, so he might choose words that may contribute to the same distribution of responses among those candidates. As a result, it's obvious that our EBTPOA is more efficient, which utilizes HCG to find distinct candidates.

V. CONCLUSIONS

Both our greedy methods, and search methods, have advantages when solving Wordle-like games. With experiments on our preliminary greedy methods, although we could not obtain the best result in the worst case, it did well in finding a strategy with low guess times on average. On the other hand, search methods could even further reduce the number of expected guess times and maximum guess times. Besides, by integrating TPOA with the heuristic of entropy, which becomes EBTPOA, we successfully improved its performance in a more efficient way. Also, we obtained the optimal result in Wordle and the best result at present in Nerdle.

ACKNOWLEDGEMENTS

This research is partially supported by the Ministry of Science and Technology (MOST) of Taiwan under Grant Numbers 110-2634-F-A49-004, 110-2221-E-A49-067-MY3 and 111-2221-E-A49-101-MY2, and the computing resources are partially supported by National Center for High-performance Computing (NCHC) of Taiwan.

REFERENCES

- [1] "Wordle - a daily word game." [Online]. Available: <https://www.nytimes.com/games/wordle/index.html>
- [2] "The daily numbers game." [Online]. Available: <https://nerdlegame.com/>
- [3] S.-T. Chen, S.-S. Lin, and L.-T. Huang, "A two-phase optimization algorithm for mastermind," *The Computer Journal*, vol. 50, no. 4, pp. 435–443, 2007.
- [4] *Solving Wordle using information theory*. 3Blue1Brown, Feb 2022. [Online]. Available: <https://youtu.be/v68zYyaEmEA>
- [5] J. Olson, "Optimal wordle solutions." [Online]. Available: <https://jonathanolson.net/experiments/optimal-wordle-solutions>
- [6] D. Bertsimas and A. Paskov, "An exact and interpretable solution to wordle," 2022. [Online]. Available: https://auction-upload-files.s3.amazonaws.com/Wordle_Paper_Final.pdf
- [7] H.-E. Kuan, T.-C. Chen, Y.-S. Lu, Y.-C. Chen, and I.-C. Wu, "Being greedy to solve nerdle: with approaches utilizing entropy." Taiwan Computer Game Association, 2022.
- [8] C. E. Shannon, "A mathematical theory of communication," *The Bell system technical journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [9] K. Little, "Best nerdle start numbers and equations." [Online]. Available: <https://nerdschalk.com/best-nerdle-start-numbers-and-equations/#comment-195906>